Functor, Applicative, Monad

Season: 6 · FP Foundations · Category: Functional · Level: Intermediate

Intent

- Type-class dictionaries with Kind<F,A> witnesses.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Lego adapters that let you map/flatMap uniformly.

When to Use

- Pure functional style in Java 25 + Zazr.
- The design force named in Intent shows up more than once.

When Not to Use

- Throwing exceptions everywhere is 'fine' for a script.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```
package com.buildwithwaqar.patterns.functional.typeclass;

/** Higher-kinded encoding: Kind&lt;F, A&gt; ≈ F&lt;A&gt;. */
public interface Kind<F, A> {}
```

Modern Java 25 / Functional

- Primary examples use Zazr; VAVR shown in comparison episode.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/typeclass/
```

Pitfalls

- Zazr rejects null inside Option/collections.
- Keep interpreters at the edge.

Interview Q&A

- Q: Interview: what problem does Functor, Applicative, Monad solve? A: Type-class dictionaries with Kind<F,A> witnesses.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Type-class dictionaries with Kind<F,A> witnesses.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Lego adapters that let you map/flatMap uniformly. [0:40] Intent — State the design force in one sent...